Chapter 18: Agile Architecture

Study Guide, based on Mastering the TOGAF Standard by Eric Jager (2025), Chapter 18 (pp. 725-756)

Overview

Chapter 18 (pp. 725-756) argues that "agile architecture is not an oxymoron" and shows how TOGAF supports agile delivery through iteration, partitioning, and "just enough" architecture (p. 725). Section 18.1 defines agility and the basic concepts: partitions, sprints, Minimum Viable Architecture, and parallel work (pp. 726-731). Section 18.2 revisits the three levels of granularity: Strategic, Segment, Capability (pp. 731-735). Section 18.3 shows how partitions, including partitioned Transition Architectures, map to sprints (pp. 735-739). Section 18.4 maps ADM phases to agile and SAFe concepts and adapts each phase for sprints (pp. 739-754). Section 18.5 summarizes (pp. 754-756). Leave with this idea: partition the architecture into Minimum Viable Architectures that teams can build in parallel sprints, while keeping the strategic big picture (pp. 728-729, 736). Chapter 18 is part of the BLUE reading tour (p. 725).

> **Reading note:** The definitions of agile, Minimum Viable Architecture, Transition Architecture, and Minimum Viable Product, and the granularity-to-agile mapping, come from the Standard and its Series Guides (G20F "Enabling Enterprise Agility", G210 "Applying the TOGAF ADM using Agile Sprints"). The painting analogy (p. 731) and the plain-language summaries are Jager's. Don't confuse **Minimum Viable Architecture** (the least architecture that is realizable and adds value) with **Minimum Viable Product** (the least output meeting minimum requirements in live operation).

Detailed Section Breakdown

Chapter Opener (p. 725)

- Agile architecture embraces change, collaboration, and incremental value. TOGAF adapts to it through iterative cycles, continuous stakeholder engagement, and incremental delivery, and even *enhances* agile by managing complexity and change in a structured way (p. 725).

18.1 What Is Agile Architecture (pp. 726-731)

- **Agility.** Adaptability. Short iterative cycles. Reprioritization toward high-value outcomes with minimal intermediate documentation. Experimentation ("fail fast"). Autonomous multidisciplinary teams. "A journey, not a destination" (p. 726).
- **Definition.** "Agile means to move/change quickly and easily, often to provide value-generating outcomes." Organizational agility is broader than the Agile Software Development Manifesto (p. 726).

18.1.1 The Basic Concepts Explained (pp. 727-731)

Concept	Meaning	Page
Architecture partition	A logical scope of work that can be independently architected and managed (Ch. 13.4)	p. 727
Sprint	A short, time-bound iteration that delivers working functionality. Each partition maps to one or more sprints	p. 728
Minimum Viable Architecture	"The minimum architecture that is realizable and add[s] business value". A partition contains at least this	p. 728
Parallel work	Agile does not require sequential completion, but the big picture must be kept	p. 729

- **Regular approach (Figure 18-3) mapped to the ADM.** Define Problem (A), Define Baseline and Define Target (B-D), Deploy Target (E-G), Govern and Manage Change (H) (pp. 729-730).
- **With sprints (Figure 18-4).** Each component runs several, possibly parallel, sprints. Define *enough* of the problem to give context and refine it as you go, but no part of the target can be implemented before its baseline and target are sufficiently defined (pp. 730-731).
- **TOGAF's route to agile** is architecture partitioning across teams with clear definitions, delivered via sprints (p. 731).

Jager's commentary (author's view): "It's like creating a painting. If you don't know what you want the painting to look like, you cannot start painting" (p. 731).

18.2 Three Levels of Architecture Granularity (pp. 731-735)

Level	Description (Ch. 7.2)	Agile role	Page
Strategic Architecture	Long-term summary of the whole organization, for executive direction	A dynamic iteration, not a static blueprint. Supported by A (and detailed in B). Identifies needed architecture and delivery capabilities. The problem is defined here and at segment level	pp. 731-733
Segment Architecture	Detailed operating models for areas. Program or portfolio level ("Domain Architecture")	"Just enough": key features plus functional and non-functional requirements, without every ADM step. Parallel iterations by teams, who help define it	pp. 731-733
Capability Architecture	Detail on supporting a capability, with increments, grouping work packages	Link between strategy and implementation. Increments map to one or more sprints. Builds on Segment Architectures. The target is deployed here by agile teams	pp. 732, 734-735

- **Example (Figures 18-6, 18-7).** A Capability Architecture within *Information Management* is partitioned into three components (a logical application component, an application service, a data entity), each built in its own sprint (pp. 734-735).

18.3 Architecture Partitions and Agile Sprints (pp. 735-739)

- **No waiting.** Segment work starts once the areas of interest are known, and Capability work doesn't wait for Segments to finish (pp. 735-736).
- **Balance detail.** Provide enough detail at each level, but not too much too early without feedback from earlier sprints. Partitions are "the agile equivalent of a Minimum Viable Architecture" (p. 736).
- **Flow.** Identify segments, refine into Capability Architectures, deliver incrementally (in parallel where dependencies allow). The output is a solution specification built by agile teams under architecture guidelines, metrics, and compliance (p. 736).
- **Partitioned transitions (Figures 18-8 to 18-11).** A single Transition Architecture plateau can be split into Transitions A, B, C, each developed in sprints. A Transition Architecture is "a formal description of one state of the architecture at an architecturally significant point in time, usually including a number of capability increments" (pp. 737-739).

18.4 Developing an Agile Architecture (pp. 739-754)

- The three levels and their ADM phases map well to agile, and each phase supports agility (pp. 739-740).

18.4.1 Mapping the ADM to Agile Concepts (pp. 740-744)

Granularity	ADM phases	SAFe / Agile / Scrum layer
Strategic	Preliminary, A	SAFe Strategic Themes. Enterprise Portfolio
Segment	B, C, D	Product/Portfolio Backlog. Change Plans. Epics
Capability	E, F, G, H	Product/Solution Backlog (capabilities). Program Backlog (capability increments, i.e., features). Program Team and Scrum Sprint Team Backlogs (deliverable user stories)

(Table 18-1, pp. 740-741)

- **Strategic level.** Preliminary (if capability changes are needed) and A produce strategic plans called *courses of action*. Agile equivalents are strategic themes and the top portfolio backlog. Cross-functional teams are needed (p. 741).
- **Segment level.** Partition courses of action across units. Explore B-D further if A is insufficient. Self-organizing teams and a high-level C/D iteration produce *Epics* and an initial segment backlog, which can feed prototypes (pp. 741-742).
- **Capability level.** B-D specify ABBs (functional and non-functional). E-F develop SBBs and their integration. The smallest deliverable unit is the *Minimum Viable Product*: "an output that satisfies a minimum set of functional and non-functional requirements and can be realized when implemented in a live operational environment" (p. 742).
- **Backlogs are refined into sprint backlogs** deliverable within weeks or a month. G confirms that contracted capabilities were delivered and that operating information is stored. H confirms that benefits were realized and users are satisfied (pp. 743-744).

18.4.2 ADM Phases and Agile Sprints (pp. 744-754)

- **Each iteration should yield a Minimum Viable Architecture.** Figure 18-13 shows the per-phase sprint adaptation (the Preliminary Phase is omitted) (pp. 744-745).
- **Phase A.** A high-level vision of outcomes, capabilities, and value. It evolves gradually ("evolutionary, not revolutionary"), is a stable foundation for growth, and gives teams clear direction (p. 745).
- **Phases B-D.** Break the work into Minimum Viable Architectures delivered by parallel teams. Business Architects, Enterprise Architects, and solution teams collaborate on Minimum Viable Business Designs. B-E need no strict order (pp. 745-747).

Phase E adapted for sprints (Table 18-2)	Phase F adapted for sprints (Table 18-3)
Understand stakeholder readiness (may need doing only once)	Confirm the management framework (before the agile journey)
Identify business constraints before each sprint, and review after a series	Assign business value to work packages per sprint
Review the gap analysis regularly (Ch. 14.5)	Estimate resources and timelines during sprint planning
Manage cross-functional requirements each sprint	Prioritize by cost/benefit and risk per sprint
Ensure interoperability of sprint solutions (Ch. 14.6)	Update the roadmap and ADD after each sprint
Refine dependencies between teams	Complete the migration plan for the next sprint (update the backlog)
Assess readiness and risks for the MVP (Ch. 14.8)	Document lessons learned each sprint

Phase E adapted for sprints (Table 18-2)	Phase F adapted for sprints (Table 18-3)
Develop an implementation strategy, define work packages for backlogs, plan transitions, create the roadmap and migration plan with product owners (Ch. 12.8.3)	

(pp. 746-748)

- **Phase G for sprints (Table 18-4).** Compliance is less formal and *ongoing*, so short cycles catch non-compliance early. Scope and resources are confirmed at sprint start, guidance is informal, an increment is deployed after each approved demo, and the post-implementation review summarizes earlier reviews (pp. 749-750).
- **Phase H for sprints (Table 18-5).** Separate change management into a parallel process: establish value realization and deploy monitoring tools. Collect risks from demos and retrospectives. Move change requests into a business change backlog. Governance cascades (business governs the EA team, the EA team governs solution teams, and vice versa) (pp. 750-751).
- **Sprint rhythm.** Sprint zero lays the groundwork. Each sprint delivers a Minimum Viable Architecture. A demo shows stakeholders the progress. A retrospective reviews quality and process (velocity, estimate accuracy, requirement allocation). Plan from the highest-value backlog items. Parallel EA teams are possible. Decoupled design lets building start before the architecture is finished (pp. 751-753).
- **Sprint planning** is also the moment to sharpen outcomes, success measures, and functional and non-functional requirements (p. 753).

18.5 Summary (pp. 754-756)

- Agility is adaptability and results first. Strategic, Segment, and Capability levels. Partitioning into Minimum Viable Architectures delivered by sprints in parallel, respecting dependencies (pp. 754-755).
- Strategic: Preliminary and A (courses of action). Segment: organize plans across units (B-D if needed). Capability: B-D detailed, then E-F into SBBs. G catches non-compliance early. H runs in parallel to confirm value (pp. 755-756).

Key Terms & Definitions

Term	Definition	Page
Agile	To move or change quickly and easily, often to provide value-generating outcomes	p. 726
Architecture partition	A logical scope of work that can be independently architected and managed	p. 727
Sprint	A short, time-bound iteration that delivers working functionality	p. 728
Minimum Viable Architecture	The minimum architecture that is realizable and adds business value	p. 728
Minimum Viable Product	An output meeting a minimum set of functional and non-functional requirements, realizable in live operation	p. 742
Transition Architecture	A formal description of one architecture state at an architecturally significant point in time, usually including capability increments	p. 738
Courses of action	High-level strategic plans produced at the strategic level	p. 741
Epic	A large or long-running user story, produced at the segment level	p. 742
Sprint zero	The initial preparation sprint that lays the groundwork	p. 751

Figures & Tables Referenced

Figure / Table	What it shows	Page
Figures 18-1, 18-2	Architecture partitions. Partitions mapped to sprints	pp. 727-728
Figures 18-3, 18-4	Regular approach components, without and with sprints	pp. 729-730
Figures 18-5, 18-6, 18-7	Three granularity levels. A partitioned Capability Architecture built in three sprints	pp. 732-735
Figures 18-8 to 18-11	Baseline, Transition, Target plateaus. Partitioned transitions and sprints	pp. 737-739
Table 18-1, Figure 18-12	Granularity and ADM phases mapped to agile and SAFe	pp. 740-743
Figure 18-13	ADM phases and sprints	p. 745
Tables 18-2 to 18-5	Adapting Phases E, F, G, H for sprints	pp. 746-751

Self-Check Questions

Q: What is a Minimum Viable Architecture, and how does it relate to partitions?

A: The minimum architecture that is realizable and adds business value. Partitions are the agile equivalent of Minimum Viable Architectures (pp. 728, 736).

Q: Map the five components of the regular approach to ADM phases.

A: Define Problem: A. Define Baseline and Define Target: B-D. Deploy Target: E, F, G. Govern and Manage Change: H (pp. 729-730).

Q: Must the problem definition be complete before other work starts?

A: No. Define enough to give context and keep refining. But you can't implement a part of the target until its baseline and target are sufficiently defined (pp. 730-731).

Q: Which ADM phases and agile layers belong to the Segment level?

A: Phases B, C, D, mapped to the Product/Portfolio Backlog, Change Plans, and Epics (pp. 740-742).

Q: What is produced at the strategic level in agile terms?

A: Courses of action (high-level strategic plans), corresponding to strategic themes and the top portfolio backlog (p. 741).

Q: How does Phase G change in a sprint-based approach?

A: Compliance becomes informal and ongoing rather than a gate, so short cycles reveal non-compliant solutions early. Increments deploy after approved demos (pp. 749-750).

Q: How is Phase H adapted for agile?

A: Change management runs as a separate parallel process: value realization, monitoring tools, risks from demos and retrospectives, and change requests into a business change backlog (pp. 750-751).

Q: How can a single Transition Architecture be made agile?

A: Partition it into several Transition Architectures (A, B, C), each developed in one or more sprints (pp. 737-738).

Q: Why does decoupled design matter?

A: It lets solution teams start building before the whole architecture is finished, without waiting on each other (p. 752).

Q: Distinguish a Minimum Viable Architecture from a Minimum Viable Product.

A: A Minimum Viable Architecture is the least *architecture* that is realizable and adds value. A Minimum Viable Product is the least *output* meeting minimum functional and non-functional requirements that can run in live operation (pp. 728, 742).